Centro Universitário de Excelência
Sistemas de Informação

Implementação de Biblioteca de Pré-processamento Nativo em Python

Autores: Darlan Deivid S. Silva,
Dylan C. Soares,
Leonardo F. C. Nery

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é ....

1. Introdução ao Pré-processamento:

2. Classificação das Operações:

3. Processamento de Dados Ausentes:

4. Transformadores de Escala Númerica:

5. Codificadores Categóricos:

6. Aplicação na Prática do Dataset Spotify:

7. Considerações Finais:

Introdução ao Pré-processamento

- **Gargalo de Machine Learning:** Nós identificamos que dados brutos possuem ruídos que inviabilizam modelos de recomendação.

- **Nosso Objetivo:** Construir do zero uma biblioteca em Python puro para higienizar e estruturar dados, sem usar Pandas ou Scikit-Learn.

Classificações das Operações

- **Tipos de Dados Tratados:** Nós separamos nossa arquitetura para atuar em variáveis Categóricas (textos) e Numéricas (grandezas).

- **Nossas Estratégias (Módulos):**
- *MissingValueProcessor:* Limpeza de matrizes.
- *Scaler:* Tratamento matemático de amplitudes.
- *Encoder:* Tradução de texto para vetor numérico.

Pré-processamento de Dados Nulos

- **Identificação de Ruídos:** Nós mapeamos dinamicamente os índices que contêm None.

- **Operações Implementadas:**
dropna: Exclusão de instâncias corrompidas.
fillna: Imputação estática de valores padrão.

```python
index_nulo = []

        # Percorre o dataset linha por linha
        for i in range(row_count):
            # Para cada linha, verifica as colunas alvo
            for col in target_cols:
                # Se eu encontrar um nulo nesta linha, guarda o índice e pulo para a próxima
                if self.dataset[col][i] is None:
                    index_nulo.append(i)
                    break
```

Transformadores de Escala Númerica

- **Problema:** Variáveis absolutas maiores enviesam os algoritmos.

- **Min-Max Scaler:** Nós reescalonamos valores para o intervalo proporcional de 0 a 1.

- **Standard Scaler (Z-score):** Nós centralizamos a distribuição (Média 0 e Desvio Padrão 1).

```python
valor_min = min(valores_validos)
            valor_max = max(valores_validos)

            denominador = valor_max - valor_min

            nova_lista = [] #Criar uma nova lista

            #Percorre por cada número da lista original
            for valor in dados_coluna:

                if valor is None:
                    nova_lista.append(None) #Deixa o vazio como vazio
                else:
                    if denominador == 0:
                        novo_valor = 0.0
                    else:
                        novo_valor = (valor - valor_min) / denominador
```

Codificadores Categóricos

- **Problema:** Máquinas não processam texto bruto.

- **Label Encoding:** Nós mapeamos valores únicos para IDs inteiros sequenciais.

- **One-Hot Encoding:** Nós geramos colunas binárias independentes (variáveis *dummy*) e removemos a raiz original.

```python
valores_unicos = []
            for valor in dados_coluna:
                if valor not in valores_unicos:
                    valores_unicos.append(valor)

            # 2. Atribui um número para cada categoria de texto
            mapeamento = {valor: indice for indice, valor in enumerate(valores_unicos)}

            # 3. Substitui os valores antigos pelos novos números na coluna
            self.dataset[col] = [mapeamento[valor] for valor in dados_coluna]

```

Aplicação no Dataset Spotify

- **Execução do Pipeline:** Validação em 8.582 linhas reais.
- **Nossos Resultados:**
  - Reduzimos para 5.221 músicas limpas (dropna).
  - A 'Popularidade' musical atingiu a escala exata de 0 a 1 (minMax_scaler).
  - Tipos de álbuns convertidos para matrizes inteiras (0 e 1).

```
PS C:\Users\leone\Documents\GitHub\OAT's\Pre_Processamento_Dados\dende-softhouse-preprocessing
_vitoria> python analise_spotify.py

==============================
 Iniciando pipeline de Pré-processamento (Spotify)
================================================

Sucesso! 15 colunas carregadas.
1. Total de músicas (linhas) originais no CVS: 8582
Músicas que contêm dados sujos (N/A): 3361
Restaram 5221 músicas limpas para a análise.
Valor máximo antigo da popularidade: 99
Nova escala criada! Mínimo: 0.0 | Máximo: 1.0
Primeiros tipos de álbum (Texto): ['album', 'single', 'album', 'single', 'single']
Textos convertidos para IDs numéricos: [0, 1, 0, 1, 1]

================================================
 PIPELINE CONCLUÍDO COM SUCESSO!
================================================
```

Considerações finais

- **Desafios:** Manipulação de laços e dicionários sem a biblioteca Pandas.
- **Conquistas:** 100% de aprovação na nossa suíte de testes unitários.
- **Trabalhos Futuros:** Otimizar a complexidade temporal $O(n)$ para cenários massivos de *Big Data*.

Referências

•Faceli, K., et al. (2021). Inteligência Artificial. LTC.

•Han, J., Kamber, M., & Pei, J. (2011). Data Mining. Morgan Kaufmann.

•Python Software Foundation (2024) https://docs.python.org/3/.